



東北大學

机器学习与 Python 实践

第五次作业

基于 Auto 数据集的汽车款式聚类分析

姓名：

专业： 强基班

学号：

教师：

学院： 信息科学与工程学院

2026 年 6 月 15 日

目录

1	前言及问题概述	2
2	相关基础知识及方法	2
2.1	相关原理	2
2.2	数据描述	3
2.3	算法实现	6
3	实验结果与可扩展内容	6
3.1	实验结果	6
3.2	参数调优及可扩展内容	11
4	讨论与结论	11
4.1	结果讨论	11
4.2	结论总结	12
5	心得体会与思考	12
5.1	个人心得	12
5.2	启发与思考	12
6	附录	13
	算法代码及其注释	13
	关键运行结果汇总	28

1 前言及问题概述

汽车市场中的车型在燃油经济性、发动机规格、整车重量、动力性能和生产年代等方面存在明显差异。面对大量车型参数，消费者往往难以仅凭单个指标判断车辆之间的相似性。聚类分析能够在没有人工类别标签的情况下，根据样本间的特征距离自动发现数据内部结构，将参数相近的汽车划分到同一组，为车型定位、市场细分和购车比较提供参考。

本次实验使用题目给出的 ISLR Auto 数据集。数据集共包含 392 辆汽车和 8 个特征，分别为每加仑汽油可行驶英里数 mpg、气缸数 cylinders、发动机排量 displacement、发动机马力 horsepower、重量 weight、从 0 加速至 60 mph 所需时间 acceleration、生产年份 year 和生产地 origin。其中 origin 的取值 1、2、3 分别表示美国、欧洲和日本。

题目要求根据汽车参数进行聚类，识别相似汽车。本实验属于无监督学习任务，不存在预先给定的真实类别。实验首先完成数据质量检查和探索性分析，再对连续变量进行标准化、对生产地进行 One-Hot 编码；随后扫描不同聚类数，使用肘部法、轮廓系数、Calinski-Harabasz 指数和 Davies-Bouldin 指数选择合理的聚类数；最后比较 K-Means、Ward 层次聚类、Gaussian Mixture 和 DBSCAN，并通过 PCA 降维图、轮廓图和聚类画像解释汽车款式差异。

2 相关基础知识及方法

2.1 相关原理

聚类的目标是在没有类别标签的条件下，将样本集合划分为若干组，使同一组内的样本尽可能相似，不同组之间的样本尽可能不同。设第 i 辆汽车的特征向量为 x_i ，聚类结果为 $c_i \in \{1, \dots, K\}$ ，则聚类算法需要根据特征空间中的距离或概率分布确定 c_i 。

K-Means 是最常用的划分式聚类方法。给定聚类数 K ，算法通过反复执行“分配样本”和“更新中心”两个步骤，最小化簇内平方和：

$$J = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|_2^2, \quad (1)$$

其中 C_k 表示第 k 个簇， μ_k 为该簇的中心。K-Means 计算速度快、结果容易解释，但需要预先指定 K ，并且对变量尺度和初始中心较敏感。

层次聚类从每个样本各自成簇开始，逐步合并距离最近的两个簇，最终形成树状层次结构。本实验采用 Ward 连接准则，每次选择使簇内平方和增加最少的合并方案。该方法不依赖随机初始化，适合探索中小规模数据的层次结构。

Gaussian Mixture 将数据看作多个高斯分布的混合，通过期望最大化算法估计每个分量的均值、协方差和混合权重。与 K-Means 的“硬划分”不同，它能够给出样本属

于各簇的概率。DBSCAN 则根据局部密度形成簇，不需要预先指定簇数，并可将稀疏区域样本标记为噪声；但其结果对邻域半径 `eps` 和最小样本数 `min_samples` 较敏感。

由于聚类没有真实标签，本实验采用内部评价指标。样本 i 的轮廓系数定义为

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}, \quad (2)$$

其中 $a(i)$ 为样本与同簇其他样本的平均距离， $b(i)$ 为样本与最近其他簇的平均距离。 $s(i)$ 越接近 1，说明样本与本簇更紧密、与其他簇分离更清楚。Calinski-Harabasz 指数越大越好，Davies-Bouldin 指数越小越好。实验还使用调整兰德指数（ARI）衡量不同算法或不同随机种子结果的一致性。

主成分分析（PCA）通过线性变换将原始高维特征投影到少数互相正交的主成分上。第一主成分保留最大方差，第二主成分在与第一主成分正交的条件下保留次大方差。本实验使用 PCA 将预处理后的 10 维数据投影到二维平面，仅用于聚类结果可视化，不参与聚类模型训练。

2.2 数据描述

数据文件为 `数据集.csv`，读取后共有 392 行、8 列。所有字段均可成功转换为数值类型，缺失值总数为 0，完全重复记录为 0，因此不需要删除样本或填补缺失值。主要字段含义见表 1。

表 1: Auto 汽车数据集字段说明

字段	类型	含义
<code>mpg</code>	连续特征	一加仑汽油支持行驶的英里数
<code>cylinders</code>	离散数值特征	发动机气缸数
<code>displacement</code>	连续特征	发动机排量
<code>horsepower</code>	连续特征	发动机马力
<code>weight</code>	连续特征	汽车重量
<code>acceleration</code>	连续特征	从 0 加速到 60 mph 所需时间
<code>year</code>	离散数值特征	车型年份，70 表示 1970 年
<code>origin</code>	类别特征	生产地：1 美国、2 欧洲、3 日本

表 2 给出了主要统计量。车辆平均燃油经济性为 23.45 mpg，平均马力为 104.47，平均重量为 2977.58。样本中有 199 辆四缸车、103 辆八缸车和 83 辆六缸车，另有少量三缸和五缸车型；生产地方面，美国车 245 辆、欧洲车 68 辆、日本车 79 辆。

表 2: Auto 数据集描述性统计

特征	均值	标准差	最小值	中位数	最大值
mpg	23.45	7.81	9.00	22.75	46.60
cylinders	5.47	1.71	3.00	4.00	8.00
displacement	194.41	104.64	68.00	151.00	455.00
horsepower	104.47	38.49	46.00	93.50	230.00
weight	2977.58	849.40	1613.00	2803.50	5140.00
acceleration	15.54	2.76	8.00	15.50	24.80
year	75.98	3.68	70.00	76.00	82.00

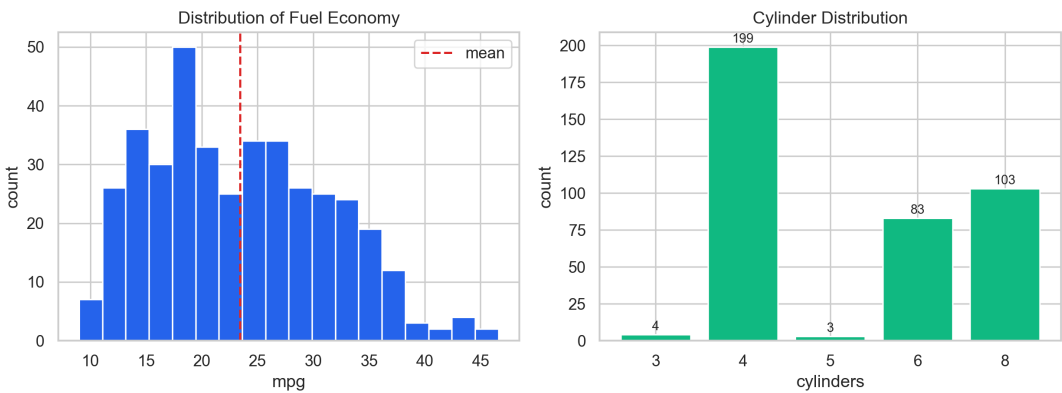


图 1: 燃油经济性与气缸数分布

从图 1 可见，mpg 分布范围较宽，四缸汽车数量最多。数值特征相关性热力图见图 2。排量、气缸数、马力和重量之间具有较强正相关，而这些动力与尺寸指标均与 mpg 呈明显负相关。例如，重量与燃油经济性的相关系数约为 -0.83 ，说明重型车辆通常耗油更多。这种成组相关关系意味着汽车样本可能沿“轻量节能”和“重型高功率”方向形成自然结构。

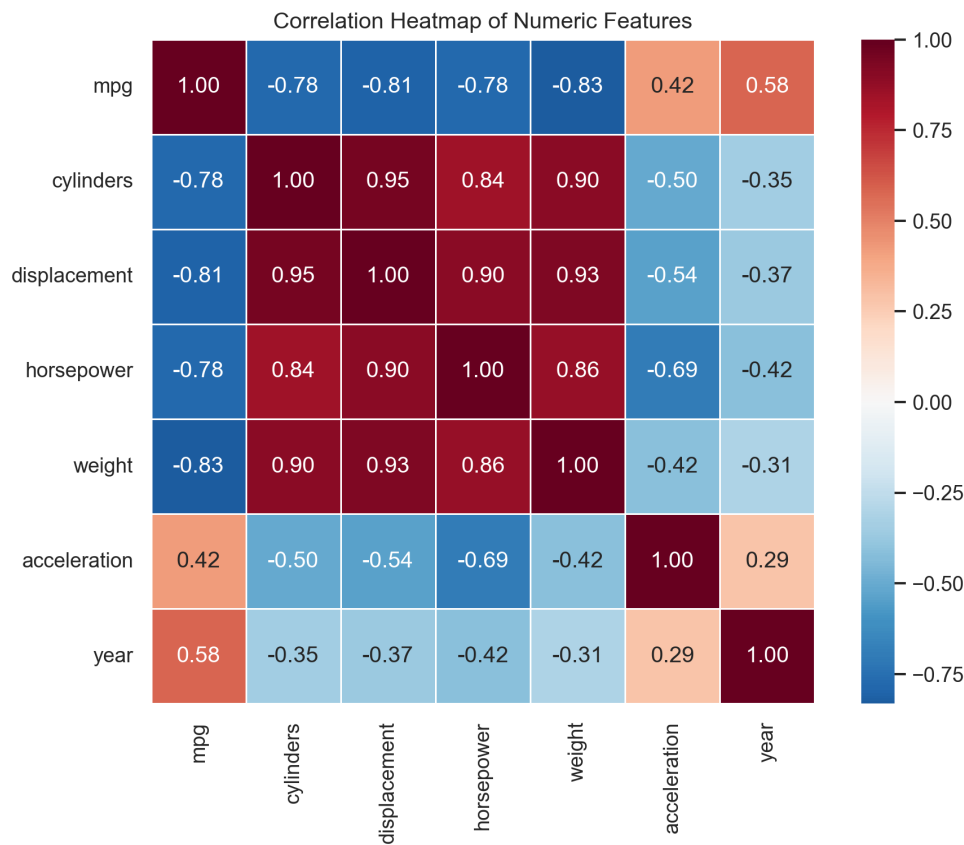


图 2: 汽车数值特征相关性热力图

图 3 进一步展示生产地、重量与燃油经济性的关系。欧洲车和日本车整体更集中在轻量、高 mpg 区域，美国车覆盖范围更广，并包含较多重量大、燃油经济性低的车型。生产地可能与车型设计取向有关，但生产地本质上是名义类别，不能直接假设“日本与欧洲的距离小于日本与美国”。因此实验将 origin 进行 One-Hot 编码。

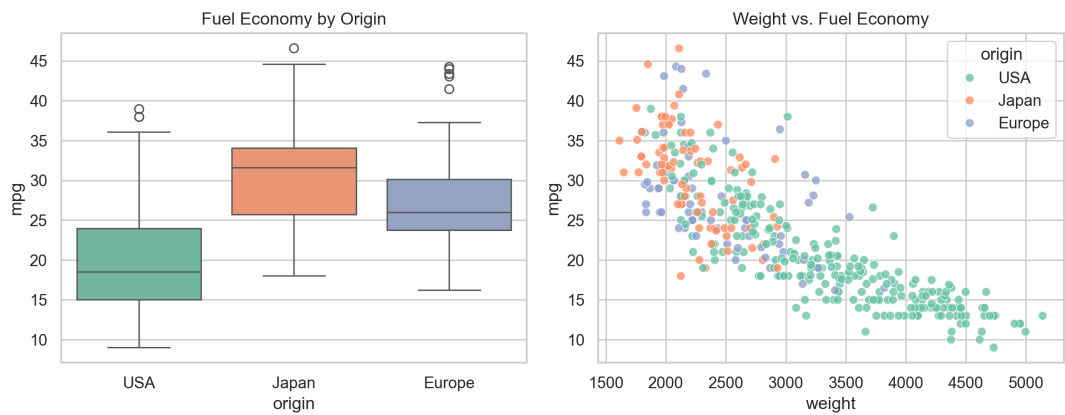


图 3: 不同生产地的燃油经济性 & 重量-mpg 关系

2.3 算法实现

实验程序使用 `pandas` 读取和整理数据，使用 `scikit-learn` 完成预处理、聚类、指标评价和 PCA 降维，使用 `matplotlib` 与 `seaborn` 绘图。具体流程如下：

1. 读取数据集 `.csv`，检查字段、数据类型、缺失值、重复值和描述性统计。
2. 保存前 10 行数据、字段类型、缺失值统计、描述性统计、生产地分布和气缸数分布。
3. 绘制燃油经济性分布、气缸数分布、相关性热力图，以及生产地、重量和 mpg 的关系图。
4. 对 `mpg`、`cylinders`、`displacement`、`horsepower`、`weight`、`acceleration` 和 `year` 使用 Z-score 标准化；对 `origin` 使用 One-Hot 编码。预处理后共有 10 个输入维度。
5. 对 K-Means 扫描 $K = 2, \dots, 8$ ，记录簇内平方和、轮廓系数、Calinski-Harabasz 指数、Davies-Bouldin 指数和簇规模。
6. 在选定的聚类数下比较 K-Means、Ward 层次聚类和 Gaussian Mixture；同时在参数网格中搜索 DBSCAN，并记录噪声比例。
7. 对 K-Means 使用 20 组不同随机种子重新训练，以 ARI 检查结果稳定性。
8. 选择内部指标表现较好的模型作为最终模型，输出聚类后的完整数据、各簇规模、原始特征均值、生产地比例和气缸数比例。
9. 使用 PCA 二维投影、轮廓图、标准化聚类画像雷达图和重量-mpg 散点图解释聚类结果。

完整 Python 程序见附录。

3 实验结果与可扩展内容

3.1 实验结果

K-Means 聚类数扫描结果见图 4 和表 3。当 K 从 2 增加至 3 时，簇内平方和从 1447.46 降至 1087.16，之后仍会继续下降，这是增加聚类数的自然结果。但 $K = 2$ 的轮廓系数最高，为 0.4347，同时 Davies-Bouldin 指数最低，为 0.8318；从 $K = 3$ 开始，轮廓系数明显下降。因此，本实验选择两个主要汽车款式。

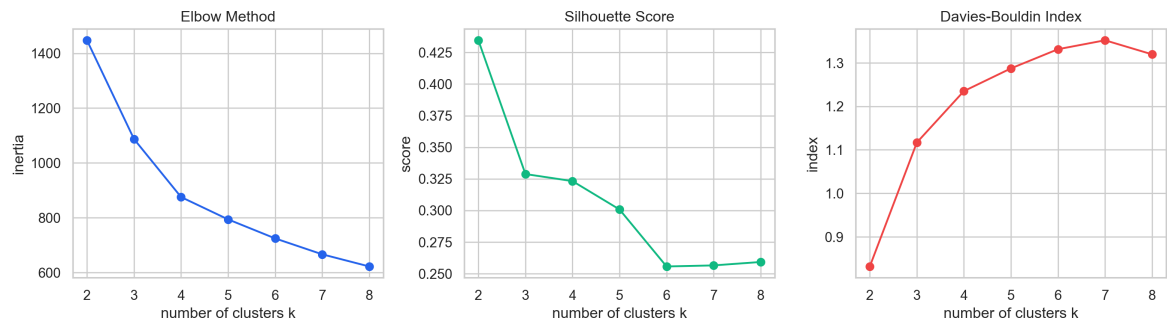


图 4: K-Means 聚类数选择结果

表 3: 不同聚类数下的 K-Means 内部评价指标

<i>K</i>	簇内平方和	轮廓系数	Calinski–Harabasz	Davies–Bouldin
2	1447.46	0.4347	406.23	0.8318
3	1087.16	0.3288	334.19	1.1167
4	875.71	0.3232	307.11	1.2349
5	794.19	0.3008	263.25	1.2872
6	724.82	0.2556	237.55	1.3313
7	666.87	0.2565	220.18	1.3518
8	622.52	0.2592	205.56	1.3195

表 4 给出了不同算法的比较。Ward 层次聚类的轮廓系数最高，为 0.4558，Davies–Bouldin 指数也最低，为 0.7469，因此将其作为最终聚类模型。K-Means 的轮廓系数为 0.4347，虽然略低于层次聚类，但 Calinski–Harabasz 指数最高；两种算法的 ARI 为 0.7711，说明它们对汽车总体结构的判断较为一致。Gaussian Mixture 的轮廓系数仅为 0.2343。DBSCAN 在最优搜索参数 `eps=0.9`、`min_samples=10` 下形成 6 个簇，但将 33.16% 的样本判为噪声，其轮廓系数只在非噪声样本上计算，不能与其余算法完全等价比较。

表 4: 不同聚类算法的内部评价结果

算法	簇数	噪声比例	轮廓系数	Davies–Bouldin	与 K-Means 的 ARI
Ward 层次聚类	2	0	0.4558	0.7469	0.7711
K-Means	2	0	0.4347	0.8318	1.0000
DBSCAN	6	0.3316	0.4138	0.9594	0.2665
Gaussian Mixture	2	0	0.2343	1.3791	0.1332

此外，K-Means 在 20 组不同随机种子下与参考结果的平均 ARI 为 0.9927，标准差为 0.0099，最小值仍达到 0.9792，说明两类结构具有较高稳定性。尽管最终模型采

用不依赖随机初始化的 Ward 层次聚类，K-Means 稳定性实验仍从另一角度验证了数据中的主要分组不是由一次随机初始化偶然产生的。

最终模型将 392 辆汽车划分为两类：292 辆“节能轻量型”，占 74.49%；100 辆“高功率重型”，占 25.51%。两类原始特征均值见表 5。

表 5: 最终聚类的汽车款式画像

类别	mpg	气缸	排量	马力	重量	加速时间	年份	生产地均值
节能轻量型	26.45	4.61	142.68	85.32	2585.81	16.51	76.75	1.77
高功率重型	14.68	7.98	345.47	160.40	4121.56	12.70	73.74	1.00

“节能轻量型”平均燃油经济性为 26.45 mpg，平均重量为 2585.81，平均马力为 85.32；其中 68.15% 为四缸车，28.08% 为六缸车，生产地构成为美国 49.66%、欧洲 23.29%、日本 27.05%。这一类车型整体较轻、排量和马力较小、生产年份较新，燃油经济性更好。

“高功率重型”平均燃油经济性仅为 14.68 mpg，平均重量达到 4121.56，平均马力为 160.40；其中 99% 为八缸车，1% 为六缸车，并且全部来自美国。该类车辆平均从 0 加速到 60 mph 的时间为 12.70，低于节能轻量型的 16.51，说明其虽然车身更重、油耗更高，但动力与加速性能更强。生产年份均值为 73.74，也表明该类车型更集中于较早年代。

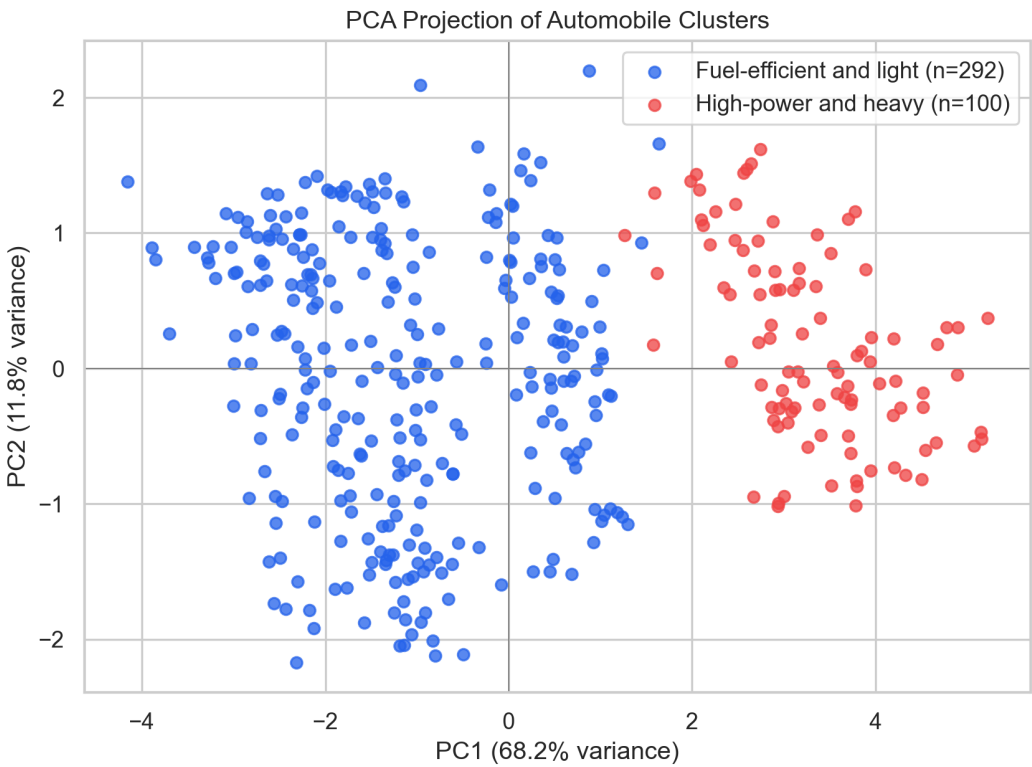


图 5: 最终聚类结果的 PCA 二维投影

图 5 中，第一主成分解释 68.2% 的方差，第二主成分解释 11.8%，两者合计解释 79.9% 的信息。两类汽车主要沿第一主成分方向分离，高功率重型车辆集中在右侧，节能轻量型车辆分布在左侧。少量样本位于边界附近，说明中等重量、中等排量的汽车仍具有连续过渡，而不是完全孤立的两个群体。

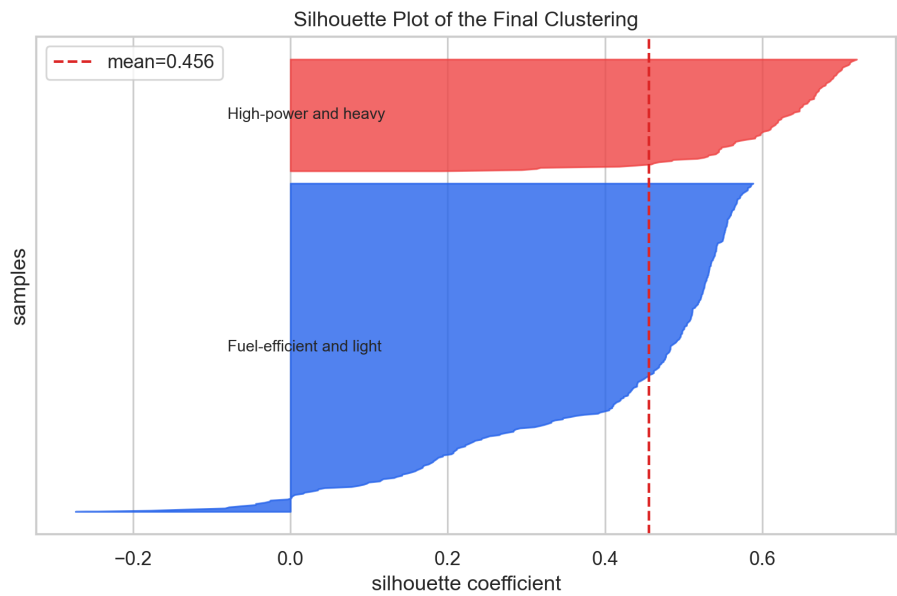


图 6: 最终层次聚类的轮廓系数分布

轮廓图见图 6。平均轮廓系数为 0.4558，大多数样本轮廓系数为正，说明总体分组较合理；少量靠近零或为负的样本多处于两类车型的过渡区域。对于真实汽车数据，这种边界模糊是合理的，因为动力、重量和油耗本身是连续变化的。

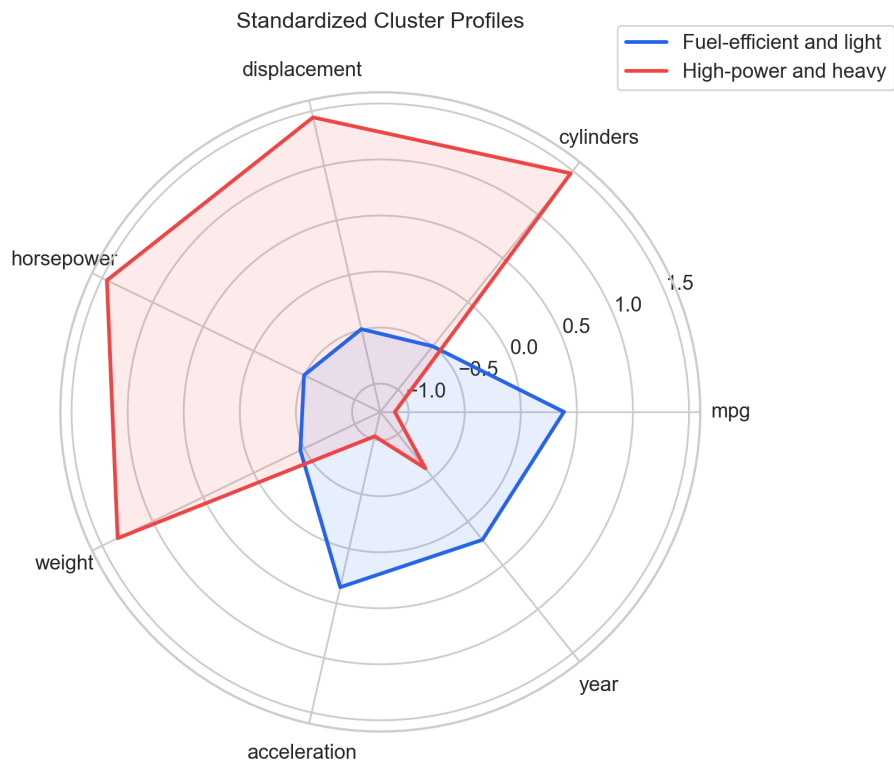


图 7: 两类汽车的标准化的特征画像

图 7 将各簇均值相对于全体数据进行标准化。高功率重型车辆在气缸数、排量、马力和重量上明显高于总体均值，在 mpg、年份和加速时间数值上低于总体均值；节能轻量型则呈现相反模式。该结果与相关性分析一致，表明聚类主要捕捉了车辆尺寸、动力和燃油经济性之间的综合差异。

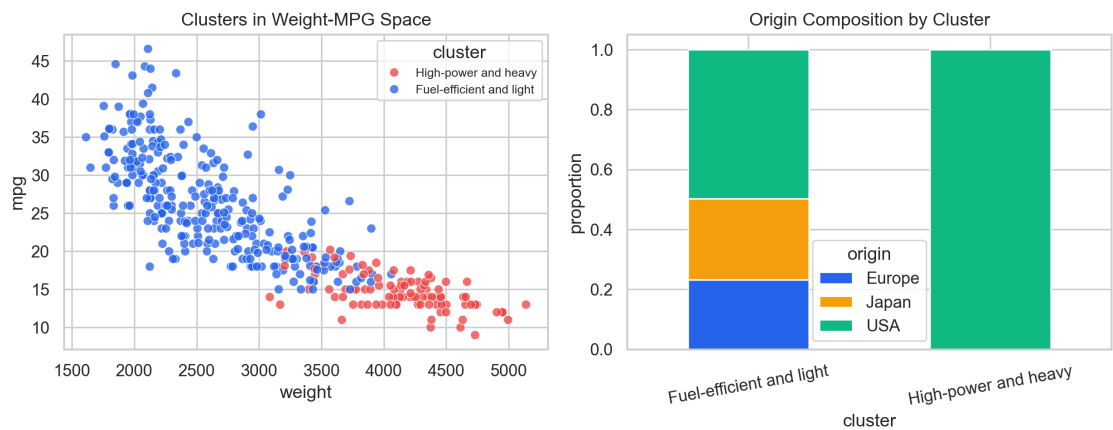


图 8: 重量-mpg 空间中的聚类及各类生产地构成

图 8 左图显示，高功率重型车辆主要集中在高重量、低 mpg 区域；右图则显示该类车型全部来自美国，而节能轻量型同时包含美国、欧洲和日本车型。这并不表示“生产地决定类别”，因为聚类同时利用了所有参数，但它揭示了本数据所覆盖年代内不同地区车型设计取向的统计差异。

3.2 参数调优及可扩展内容

本实验通过 $K = 2, \dots, 8$ 的网格扫描选择 K-Means 聚类数, 并对 DBSCAN 的 `eps` 和 `min_samples` 进行组合搜索。结果表明, 继续增加 K 虽然可以降低簇内平方和, 但会使轮廓系数下降、Davies–Bouldin 指数上升, 因此不应仅凭簇内平方和选择更复杂的划分。两个主要簇能够提供最清晰、最稳定的宏观车型分组。

若需要更细粒度的市场细分, 可以在“节能轻量型”内部再次聚类, 将小型经济车、中型均衡车和不同生产地车型进一步区分。这种分层聚类思路比直接在全体数据上强制设置较大的 K 更容易解释。还可以使用 UMAP 或 t-SNE 辅助观察非线性结构, 但这些降维方法对参数和随机种子较敏感, 不应单独作为确定聚类数的依据。

在特征处理方面, 本实验对 `origin` 使用 One-Hot 编码, 避免把生产地当作有序数值。未来可以根据实际任务调整特征权重: 如果更关注机械结构, 可降低生产地和年份的影响; 如果更关注消费市场, 可加入价格、车身尺寸、品牌、维护成本和安全配置等变量。还可以使用 Gower 距离或 k-prototypes 等适合混合数据类型的算法, 进一步减少数值特征与类别特征共同建模时的距离偏差。

4 讨论与结论

4.1 结果讨论

实验结果表明, Auto 数据集最显著的结构是“节能轻量型”与“高功率重型”的二分。两个类别不仅在重量和 mpg 上明显不同, 在气缸数、排量、马力、加速时间和生产年份上也形成一致差异。这说明聚类结果并非由某一个字段单独决定, 而是由多个相互关联的汽车参数共同形成。

Ward 层次聚类的平均轮廓系数为 0.4558, 略优于 K-Means 的 0.4347。K-Means 与层次聚类之间的 ARI 为 0.7711, 同时 K-Means 的随机种子稳定性接近 1, 说明两个算法虽然对部分边界车辆的划分不同, 但都识别出了同一主导结构。Gaussian Mixture 的表现较弱, 可能是因为标准化后的汽车数据并不满足简单的两个完整高斯分布假设。DBSCAN 则将较多样本视为噪声, 说明该数据更接近连续变化的椭球状群体, 而不是由高密度核心和稀疏间隔组成的形状。

需要注意的是, 聚类类别名称是根据特征均值进行的人工解释, 并非数据集中原有标签。“节能轻量型”和“高功率重型”表示统计意义上的相似组, 不等同于严格的汽车行业级别。生产地构成也只反映 1970 至 1982 年、该数据样本范围内的规律, 不能直接推广到当前汽车市场。

4.2 结论总结

本次实验完成了基于 ISLR Auto 数据集的汽车款式聚类分析。实验对 7 个数值特征进行标准化，对生产地进行 One-Hot 编码，并使用多种内部指标选择聚类数。结果显示， $K = 2$ 时 K-Means 的轮廓系数最高且 Davies-Bouldin 指数最低，说明两类划分最清晰。

在算法比较中，Ward 层次聚类取得 0.4558 的最高轮廓系数，因此被选为最终模型。最终得到 292 辆节能轻量型汽车和 100 辆高功率重型汽车。前者具有较高 mpg、较低重量、较小排量和较新年份，来源更加多样；后者几乎全部为美国八缸车，重量、排量和马力明显更高，但燃油经济性较低。

综合来看，聚类分析能够将多个汽车参数压缩为直观的车辆分组，帮助理解复杂数据中的总体结构。完整流程涵盖数据检查、探索性分析、特征编码、标准化、聚类数选择、算法比较、稳定性验证、降维可视化和聚类解释，体现了无监督学习任务中“先保证距离合理，再评价结构，最后结合业务含义解释”的基本思路。

5 心得体会与思考

5.1 个人心得

通过本次实验，我对监督学习和无监督学习的差异有了更直观的认识。分类任务有真实标签，可以直接计算准确率；聚类任务没有标准答案，需要同时观察内部评价指标、二维可视化和各簇特征画像。仅得到一个聚类编号并不能说明结果有意义，必须进一步回答每个簇包含哪些样本、各项参数有什么差异，以及这些差异是否符合实际常识。

我也体会到数据预处理对聚类尤其重要。汽车重量可达到数千，生产地只有 1、2、3，如果直接计算欧氏距离，重量会在距离中占据过大作用，而生产地编码还会引入不存在的顺序关系。通过标准化和 One-Hot 编码，各项特征才能在较合理的尺度上共同参与聚类。

5.2 启发与思考

聚类数并不是越多越好。更多的簇一定会降低簇内平方和，但不一定能形成更清晰、更稳定、更容易解释的结构。本实验中 $K = 2$ 的轮廓系数明显高于 $K = 3$ 至 $K = 8$ ，说明数据首先呈现的是两种宏观设计取向。若实际应用需要更细分类，应在已有大类内部继续细分，并结合业务目标确定粒度，而不是机械地追求更多类别。

此外，PCA 图让我看到聚类边界并非绝对分离。部分车辆位于两类之间，说明汽车参数存在连续过渡。实际决策中可以进一步计算样本到簇中心的距离或使用概率聚类，给出“典型程度”或不确定性，不应把每辆车视为绝对固定的款式。

6 附录

算法代码及其注释

```
# -*- coding: utf-8 -*-
"""Report 5: clustering automobile styles with the ISLR Auto dataset.

The script performs data validation, exploratory analysis, preprocessing,
cluster-number selection, algorithm comparison, cluster profiling, and saves
all tables/figures used by the LaTeX report.
"""

from __future__ import annotations

import argparse
import json
import os
import warnings
from pathlib import Path

os.environ.setdefault("OMP_NUM_THREADS", "2")

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

from sklearn.cluster import AgglomerativeClustering, DBSCAN, KMeans
from sklearn.compose import ColumnTransformer
from sklearn.decomposition import PCA
from sklearn.metrics import (
    adjusted_rand_score,
    calinski_harabasz_score,
    davies_bouldin_score,
    silhouette_samples,
    silhouette_score,
)
from sklearn.mixture import GaussianMixture
```

```

from sklearn.preprocessing import OneHotEncoder, StandardScaler

warnings.filterwarnings("ignore")

RANDOM_STATE = 42
NUMERIC_COLUMNS = [
    "mpg",
    "cylinders",
    "displacement",
    "horsepower",
    "weight",
    "acceleration",
    "year",
]
CATEGORICAL_COLUMNS = ["origin"]
ALL_COLUMNS = NUMERIC_COLUMNS + CATEGORICAL_COLUMNS
ORIGIN_NAMES = {1: "USA", 2: "Europe", 3: "Japan"}
COLORS = ["#2563EB", "#EF4444", "#10B981", "#F59E0B", "#8B5CF6"]

def ensure_dir(path: Path) -> Path:
    path.mkdir(parents=True, exist_ok=True)
    return path

def make_one_hot_encoder() -> OneHotEncoder:
    try:
        return OneHotEncoder(handle_unknown="ignore", sparse_output=False)
    except TypeError:
        return OneHotEncoder(handle_unknown="ignore", sparse=False)

def load_data(data_path: Path) -> pd.DataFrame:
    if not data_path.exists():
        raise FileNotFoundError(f"Cannot find data file: {data_path}")
    df = pd.read_csv(data_path)
    missing_columns = set(ALL_COLUMNS) - set(df.columns)
    if missing_columns:
        raise ValueError(f"Dataset is missing columns:
↪ {sorted(missing_columns)}")

```

```

df = df[ALL_COLUMNS].copy()
for column in ALL_COLUMNS:
    df[column] = pd.to_numeric(df[column], errors="coerce")
if df.isna().any().any():
    bad = df.isna().sum()
    raise ValueError(f"Numeric conversion produced missing
        ↪ values:\n{bad[bad > 0]}")
return df

def build_preprocessor() -> ColumnTransformer:
    return ColumnTransformer(
        transformers=[
            ("num", StandardScaler(), NUMERIC_COLUMNS),
            ("origin", make_one_hot_encoder(), CATEGORICAL_COLUMNS),
        ],
        remainder="drop",
    )

def save_overview(df: pd.DataFrame, out_dir: Path) -> None:
    ensure_dir(out_dir)
    df.head(10).to_csv(out_dir / "head_10.csv", index=False,
        ↪ encoding="utf-8-sig")
    df.dtypes.astype(str).rename("dtype").to_csv(
        out_dir / "dtypes.csv", encoding="utf-8-sig"
    )
    df.isna().sum().rename("missing_count").to_csv(
        out_dir / "missing_values.csv", encoding="utf-8-sig"
    )
    df.describe().T.to_csv(out_dir / "describe.csv", encoding="utf-8-sig")
    df["origin"].map(ORIGIN_NAMES).value_counts().rename("count").to_csv(
        out_dir / "origin_distribution.csv", encoding="utf-8-sig"
    )
    df["cylinders"].value_counts().sort_index().rename("count").to_csv(
        out_dir / "cylinder_distribution.csv", encoding="utf-8-sig"
    )

def plot_eda(df: pd.DataFrame, fig_dir: Path) -> None:
    ensure_dir(fig_dir)

```



```

sns.set_theme(style="whitegrid")

fig, axes = plt.subplots(1, 2, figsize=(10.5, 4.0))
axes[0].hist(df["mpg"], bins=18, color="#2563EB", edgecolor="white")
axes[0].axvline(df["mpg"].mean(), color="#DC2626", linestyle="--",
    ↪ label="mean")
axes[0].set(title="Distribution of Fuel Economy", xlabel="mpg",
    ↪ ylabel="count")
axes[0].legend()
cylinder_counts = df["cylinders"].value_counts().sort_index()
axes[1].bar(
    cylinder_counts.index.astype(str), cylinder_counts.values,
    ↪ color="#10B981"
)
axes[1].set(title="Cylinder Distribution", xlabel="cylinders",
    ↪ ylabel="count")
for i, value in enumerate(cylinder_counts.values):
    axes[1].text(i, value + 3, str(int(value)), ha="center", fontsize=9)
fig.tight_layout()
fig.savefig(fig_dir / "basic_distributions.png", dpi=240,
    ↪ bbox_inches="tight")
plt.close(fig)

corr = df[NUMERIC_COLUMNS].corr()
fig, ax = plt.subplots(figsize=(8.2, 6.5))
sns.heatmap(
    corr,
    annot=True,
    fmt=".2f",
    cmap="RdBu_r",
    center=0,
    square=True,
    linewidths=0.4,
    ax=ax,
)
ax.set_title("Correlation Heatmap of Numeric Features")
fig.tight_layout()
fig.savefig(fig_dir / "correlation_heatmap.png", dpi=240,
    ↪ bbox_inches="tight")
plt.close(fig)

```

```

origin_plot = df.copy()
origin_plot["origin_name"] = origin_plot["origin"].map(ORIGIN_NAMES)
fig, axes = plt.subplots(1, 2, figsize=(10.5, 4.2))
sns.boxplot(data=origin_plot, x="origin_name", y="mpg", ax=axes[0],
    ↪ palette="Set2")
axes[0].set(title="Fuel Economy by Origin", xlabel="origin",
    ↪ ylabel="mpg")
sns.scatterplot(
    data=origin_plot,
    x="weight",
    y="mpg",
    hue="origin_name",
    palette="Set2",
    alpha=0.78,
    ax=axes[1],
)
axes[1].set(title="Weight vs. Fuel Economy", xlabel="weight",
    ↪ ylabel="mpg")
axes[1].legend(title="origin")
fig.tight_layout()
fig.savefig(fig_dir / "origin_and_weight_mpg.png", dpi=240,
    ↪ bbox_inches="tight")
plt.close(fig)

def evaluate_labels(X: np.ndarray, labels: np.ndarray) -> dict[str, float]:
    unique = np.unique(labels)
    if len(unique) < 2 or len(unique) >= len(labels):
        return {
            "Silhouette": np.nan,
            "Calinski_Harabasz": np.nan,
            "Davies_Bouldin": np.nan,
        }
    return {
        "Silhouette": silhouette_score(X, labels),
        "Calinski_Harabasz": calinski_harabasz_score(X, labels),
        "Davies_Bouldin": davies_bouldin_score(X, labels),
    }

def scan_kmeans(X: np.ndarray, out_dir: Path, fig_dir: Path) -> pd.DataFrame:

```

```

rows = []
for k in range(2, 9):
    model = KMeans(n_clusters=k, n_init=50, random_state=RANDOM_STATE)
    labels = model.fit_predict(X)
    metrics = evaluate_labels(X, labels)
    rows.append(
        {
            "k": k,
            "Inertia": model.inertia_,
            **metrics,
            "Min_Cluster_Size": np.bincount(labels).min(),
            "Max_Cluster_Size": np.bincount(labels).max(),
        }
    )
results = pd.DataFrame(rows)
results.to_csv(out_dir / "kmeans_k_selection.csv", index=False,
    ↪ encoding="utf-8-sig")

fig, axes = plt.subplots(1, 3, figsize=(13.2, 3.8))
axes[0].plot(results["k"], results["Inertia"], marker="o",
    ↪ color="#2563EB")
axes[0].set(title="Elbow Method", xlabel="number of clusters k",
    ↪ ylabel="inertia")
axes[1].plot(results["k"], results["Silhouette"], marker="o",
    ↪ color="#10B981")
axes[1].set(title="Silhouette Score", xlabel="number of clusters k",
    ↪ ylabel="score")
axes[2].plot(
    results["k"], results["Davies_Bouldin"], marker="o", color="#EF4444"
)
axes[2].set(title="Davies-Bouldin Index", xlabel="number of clusters k",
    ↪ ylabel="index")
for ax in axes:
    ax.set_xticks(results["k"])
fig.tight_layout()
fig.savefig(fig_dir / "kmeans_k_selection.png", dpi=240,
    ↪ bbox_inches="tight")
plt.close(fig)
return results

```

```

def select_dbscan(X: np.ndarray) -> tuple[DBSCAN, np.ndarray, dict[str,
↪ float]]:
    candidates = []
    for min_samples in [5, 6, 8, 10]:
        for eps in np.arange(0.6, 1.61, 0.1):
            model = DBSCAN(eps=float(eps), min_samples=min_samples)
            labels = model.fit_predict(X)
            mask = labels != -1
            n_clusters = len(set(labels[mask]))
            noise_ratio = 1.0 - mask.mean()
            if n_clusters < 2 or mask.sum() < 0.6 * len(labels):
                continue
            score = silhouette_score(X[mask], labels[mask])
            candidates.append((score - 0.25 * noise_ratio, score,
↪ noise_ratio, model, labels))
    if not candidates:
        model = DBSCAN(eps=1.0, min_samples=8)
        labels = model.fit_predict(X)
        return model, labels, {"Silhouette": np.nan, "Noise_Ratio": 1.0}
    _, score, noise_ratio, model, labels = max(candidates, key=lambda item:
↪ item[0])
    return model, labels, {"Silhouette": score, "Noise_Ratio": noise_ratio}

def compare_algorithms(
    X: np.ndarray, n_clusters: int, out_dir: Path
) -> tuple[pd.DataFrame, dict[str, np.ndarray]]:
    labels_by_model: dict[str, np.ndarray] = {}
    labels_by_model["KMeans"] = KMeans(
        n_clusters=n_clusters, n_init=50, random_state=RANDOM_STATE
    ).fit_predict(X)
    labels_by_model["Agglomerative_Ward"] = AgglomerativeClustering(
        n_clusters=n_clusters, linkage="ward"
    ).fit_predict(X)
    labels_by_model["Gaussian_Mixture"] = GaussianMixture(
        n_components=n_clusters, covariance_type="full", n_init=20,
↪ random_state=RANDOM_STATE
    ).fit_predict(X)
    dbscan_model, dbscan_labels, dbscan_info = select_dbscan(X)
    labels_by_model["DBSCAN"] = dbscan_labels

```

```

kmeans_labels = labels_by_model["KMeans"]
rows = []
for name, labels in labels_by_model.items():
    if name == "DBSCAN":
        mask = labels != -1
        n_found = len(set(labels[mask]))
        metrics = evaluate_labels(X[mask], labels[mask]) if n_found >= 2
        ↪ else {}
        rows.append(
            {
                "Model": name,
                "Clusters": n_found,
                "Noise_Ratio": dbscan_info["Noise_Ratio"],
                "Silhouette": metrics.get("Silhouette", np.nan),
                "Calinski_Harabasz": metrics.get("Calinski_Harabasz",
                ↪ np.nan),
                "Davies_Bouldin": metrics.get("Davies_Bouldin", np.nan),
                "ARI_vs_KMeans":
                ↪ adjusted_rand_score(kmeans_labels[mask],
                ↪ labels[mask]),
                "Parameters": f"eps={dbscan_model.eps:.1f},
                ↪ min_samples={dbscan_model.min_samples}",
            }
        )
    else:
        metrics = evaluate_labels(X, labels)
        rows.append(
            {
                "Model": name,
                "Clusters": len(np.unique(labels)),
                "Noise_Ratio": 0.0,
                **metrics,
                "ARI_vs_KMeans": adjusted_rand_score(kmeans_labels,
                ↪ labels),
                "Parameters": f"k={n_clusters}",
            }
        )
comparison = pd.DataFrame(rows).sort_values("Silhouette",
    ↪ ascending=False)
comparison.to_csv(out_dir / "algorithm_comparison.csv", index=False,
    ↪ encoding="utf-8-sig")

```

```

    return comparison, labels_by_model

def kmeans_stability(X: np.ndarray, n_clusters: int, out_dir: Path) ->
    dict[str, float]:
    ↪ reference = KMeans(
        n_clusters=n_clusters, n_init=50, random_state=RANDOM_STATE
    ).fit_predict(X)
    scores = []
    for seed in range(20):
        labels = KMeans(n_clusters=n_clusters, n_init=10,
            ↪ random_state=seed).fit_predict(X)
        scores.append(adjusted_rand_score(reference, labels))
    stability = {
        "mean_ari": float(np.mean(scores)),
        "std_ari": float(np.std(scores)),
        "min_ari": float(np.min(scores)),
        "max_ari": float(np.max(scores)),
    }
    pd.Series(stability).to_csv(out_dir / "kmeans_stability.csv",
        ↪ encoding="utf-8-sig")
    return stability

def ordered_cluster_labels(df: pd.DataFrame, labels: np.ndarray) ->
    tuple[np.ndarray, dict[int, str]]:
    ↪ temp = df.copy()
    temp["raw_cluster"] = labels
    means = temp.groupby("raw_cluster")["mpg"].mean().sort_values(ascending=
        ↪ =False)
    order = means.index.tolist()
    if len(order) == 2:
        names = ["Fuel-efficient and light", "High-power and heavy"]
    else:
        names = [f"Style {i + 1}" for i in range(len(order))]
    raw_to_name = {raw: names[i] for i, raw in enumerate(order)}
    named = np.array([raw_to_name[label] for label in labels], dtype=object)
    return named, raw_to_name

def save_cluster_profiles(

```

```

    df: pd.DataFrame,
    labels: np.ndarray,
    out_dir: Path,
) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame, np.ndarray]:
    names, _ = ordered_cluster_labels(df, labels)
    clustered = df.copy()
    clustered["cluster"] = names
    order = clustered.groupby("cluster")["mpg"].mean().sort_values(ascending=
    ↪ g=False).index
    profile = clustered.groupby("cluster")[ALL_COLUMNS].mean().reindex(order)
    counts = clustered["cluster"].value_counts().reindex(order).rename("cou
    ↪ nt").to_frame()
    counts["ratio"] = counts["count"] / len(clustered)
    origin_share = pd.crosstab(
        clustered["cluster"], clustered["origin"].map(ORIGIN_NAMES),
        ↪ normalize="index"
    ).reindex(order)
    cylinder_share = pd.crosstab(
        clustered["cluster"], clustered["cylinders"], normalize="index"
    ).reindex(order)

    profile.to_csv(out_dir / "cluster_profiles.csv", encoding="utf-8-sig")
    counts.to_csv(out_dir / "cluster_sizes.csv", encoding="utf-8-sig")
    origin_share.to_csv(out_dir / "cluster_origin_share.csv",
        ↪ encoding="utf-8-sig")
    cylinder_share.to_csv(out_dir / "cluster_cylinder_share.csv",
        ↪ encoding="utf-8-sig")
    clustered.to_csv(out_dir / "clustered_auto_data.csv", index=False,
        ↪ encoding="utf-8-sig")
    return profile, counts, origin_share, names

def plot_clustering_results(
    df: pd.DataFrame,
    X: np.ndarray,
    labels: np.ndarray,
    names: np.ndarray,
    profile: pd.DataFrame,
    origin_share: pd.DataFrame,
    fig_dir: Path,
) -> dict[str, float]:

```

```

pca = PCA(n_components=2, random_state=RANDOM_STATE)
X_pca = pca.fit_transform(X)
explained = pca.explained_variance_ratio_
ordered_names = profile.index.tolist()
color_map = {name: COLORS[i] for i, name in enumerate(ordered_names)}

fig, ax = plt.subplots(figsize=(7.2, 5.4))
for name in ordered_names:
    mask = names == name
    ax.scatter(
        X_pca[mask, 0],
        X_pca[mask, 1],
        s=30,
        alpha=0.76,
        color=color_map[name],
        label=f"{name} (n={mask.sum()})",
    )
ax.axhline(0, color="gray", linewidth=0.6)
ax.axvline(0, color="gray", linewidth=0.6)
ax.set(
    title="PCA Projection of Automobile Clusters",
    xlabel=f"PC1 ({explained[0] * 100:.1f}% variance)",
    ylabel=f"PC2 ({explained[1] * 100:.1f}% variance)",
)
ax.legend()
fig.tight_layout()
fig.savefig(fig_dir / "pca_clusters.png", dpi=240, bbox_inches="tight")
plt.close(fig)

sample_scores = silhouette_samples(X, labels)
fig, ax = plt.subplots(figsize=(7.4, 5.0))
y_lower = 10
for i, raw_label in enumerate(np.unique(labels)):
    values = np.sort(sample_scores[labels == raw_label])
    size = len(values)
    y_upper = y_lower + size
    display_name = names[labels == raw_label][0]
    ax.fill_betweenx(
        np.arange(y_lower, y_upper), 0, values,
        color=color_map[display_name], alpha=0.8
    )

```



```

    ax.text(-0.08, y_lower + 0.5 * size, display_name, fontsize=9,
            ↪ va="center")
    y_lower = y_upper + 10
    avg_score = silhouette_score(X, labels)
    ax.axvline(avg_score, color="#DC2626", linestyle="--",
            ↪ label=f"mean={avg_score:.3f}")
    ax.set(title="Silhouette Plot of the Final Clustering",
            ↪ xlabel="silhouette coefficient", ylabel="samples")
    ax.set_yticks([])
    ax.legend()
    fig.tight_layout()
    fig.savefig(fig_dir / "silhouette_plot.png", dpi=240,
            ↪ bbox_inches="tight")
    plt.close(fig)

    radar_features = ["mpg", "cylinders", "displacement", "horsepower",
            ↪ "weight", "acceleration", "year"]
    standardized_profile = (profile[radar_features] -
            ↪ df[radar_features].mean()) / df[radar_features].std()
    angles = np.linspace(0, 2 * np.pi, len(radar_features),
            ↪ endpoint=False).tolist()
    angles += angles[:1]
    fig, ax = plt.subplots(figsize=(7.0, 6.0), subplot_kw={"polar": True})
    for name in ordered_names:
        values = standardized_profile.loc[name].tolist()
        values += values[:1]
        ax.plot(angles, values, linewidth=2, label=name,
            ↪ color=color_map[name])
        ax.fill(angles, values, alpha=0.10, color=color_map[name])
    ax.set_xticks(angles[:-1])
    ax.set_xticklabels(radar_features)
    ax.set_title("Standardized Cluster Profiles", pad=18)
    ax.legend(loc="upper right", bbox_to_anchor=(1.32, 1.12))
    fig.tight_layout()
    fig.savefig(fig_dir / "cluster_profile_radar.png", dpi=240,
            ↪ bbox_inches="tight")
    plt.close(fig)

    fig, axes = plt.subplots(1, 2, figsize=(10.5, 4.2))
    plot_df = df.copy()
    plot_df["cluster"] = names

```

```

sns.scatterplot(
    data=plot_df,
    x="weight",
    y="mpg",
    hue="cluster",
    palette=color_map,
    alpha=0.78,
    ax=axes[0],
)
axes[0].set(title="Clusters in Weight-MPG Space", xlabel="weight",
    ↪ ylabel="mpg")
axes[0].legend(title="cluster", fontsize=8)
origin_share.plot(kind="bar", stacked=True, ax=axes[1],
    ↪ color=["#2563EB", "#F59E0B", "#10B981"])
axes[1].set(title="Origin Composition by Cluster", xlabel="cluster",
    ↪ ylabel="proportion")
axes[1].tick_params(axis="x", rotation=10)
axes[1].legend(title="origin")
fig.tight_layout()
fig.savefig(fig_dir / "cluster_interpretation.png", dpi=240,
    ↪ bbox_inches="tight")
plt.close(fig)

return {
    "pca_pc1_ratio": float(explained[0]),
    "pca_pc2_ratio": float(explained[1]),
    "pca_total_ratio": float(explained.sum()),
    "final_silhouette": float(avg_score),
}

def run_experiment(data_path: Path, output_root: Path) -> dict[str, object]:
    overview_dir = ensure_dir(output_root / "results_auto_clustering" /
    ↪ "01_data_overview")
    model_dir = ensure_dir(output_root / "results_auto_clustering" /
    ↪ "02_clustering")
    fig_dir = ensure_dir(output_root / "figures")

    df = load_data(data_path)
    save_overview(df, overview_dir)
    plot_eda(df, fig_dir)

```

```

preprocessor = build_preprocessor()
X = preprocessor.fit_transform(df)
k_results = scan_kmeans(X, model_dir, fig_dir)
best_k = int(k_results.loc[k_results["Silhouette"].idxmax(), "k"])

comparison, labels_by_model = compare_algorithms(X, best_k, model_dir)
stability = kmeans_stability(X, best_k, model_dir)
final_model_name = comparison.loc[
    comparison["Model"] != "DBSCAN", "Silhouette"
].idxmax()
final_model_name = str(comparison.loc[final_model_name, "Model"])
final_labels = labels_by_model[final_model_name]

profile, counts, origin_share, names = save_cluster_profiles(df,
    ↪ final_labels, model_dir)
plot_summary = plot_clustering_results(
    df, X, final_labels, names, profile, origin_share, fig_dir
)

summary = {
    "rows": int(df.shape[0]),
    "columns": int(df.shape[1]),
    "missing_values": int(df.isna().sum().sum()),
    "duplicates": int(df.duplicated().sum()),
    "transformed_features": int(X.shape[1]),
    "best_k": best_k,
    "final_model": final_model_name,
    "final_cluster_sizes": {str(k): int(v) for k, v in
        ↪ counts["count"].items()},
    "kmeans_stability": stability,
    **plot_summary,
}

with (model_dir / "summary.json").open("w", encoding="utf-8") as file:
    json.dump(summary, file, ensure_ascii=False, indent=2)

print("Experiment completed.")
print(json.dumps(summary, ensure_ascii=False, indent=2))
print("\nAlgorithm comparison:")
print(comparison.round(4).to_string(index=False))
print("\nFinal cluster profiles:")

```

```
print(profile.round(2).to_string())
return summary

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Auto dataset clustering
↪ experiment")
    parser.add_argument("--data", type=Path, default=Path("数据集.csv"))
    parser.add_argument("--output-root", type=Path, default=Path("."))
    return parser.parse_args()

if __name__ == "__main__":
    args = parse_args()
    run_experiment(args.data, args.output_root)
```

关键运行结果汇总

数据规模: 392 行, 8 列

缺失值: 0

重复记录: 0

预处理后特征维度: 10

最优主要聚类数: 2

最终模型: Agglomerative_Ward

最终轮廓系数: 0.455824

聚类规模:

Fuel-efficient and light: 292

High-power and heavy: 100

K-Means 20 组随机种子稳定性:

平均 ARI: 0.992732

标准差: 0.009904

最小 ARI: 0.979235

PCA 解释方差:

PC1: 68.19%

PC2: 11.76%

合计: 79.95%